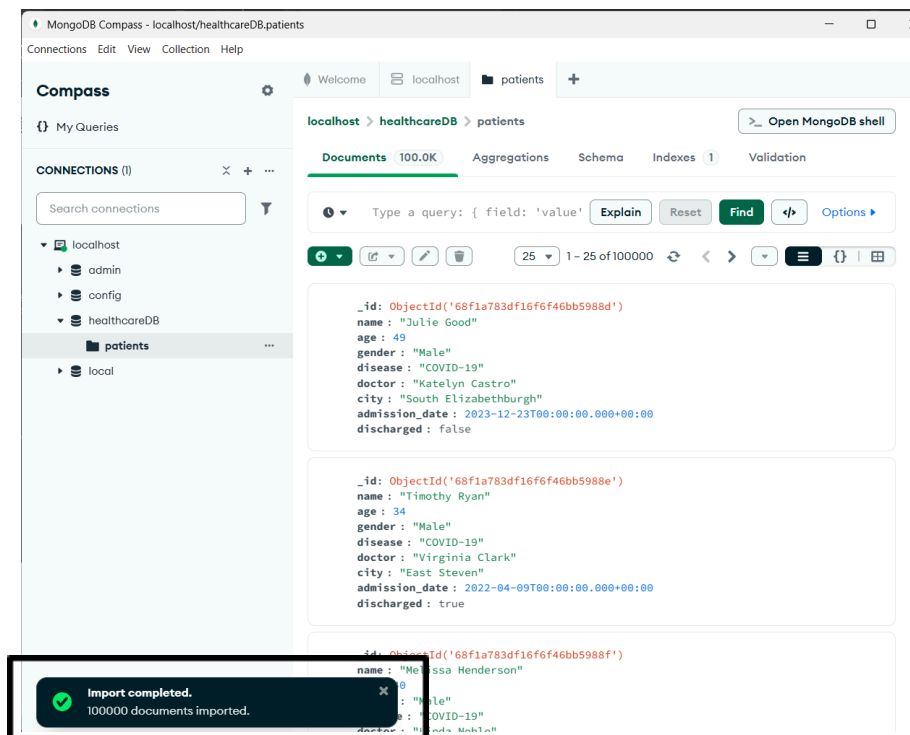


DBMD Assignment: Explore MongoDB for CRUD operations And MySQL Tasks

Submitted By: 25MCB1010, Patsamatla Bhaavya Ramita

1. MongoDB Database creation: Created a healthcare database and patients collections to store patient records.
2. Data Insertion: Faker created 100,000 patient records using a csv file with attributes: name, age, gender, disease, doctor, city, admission_date, discharged.

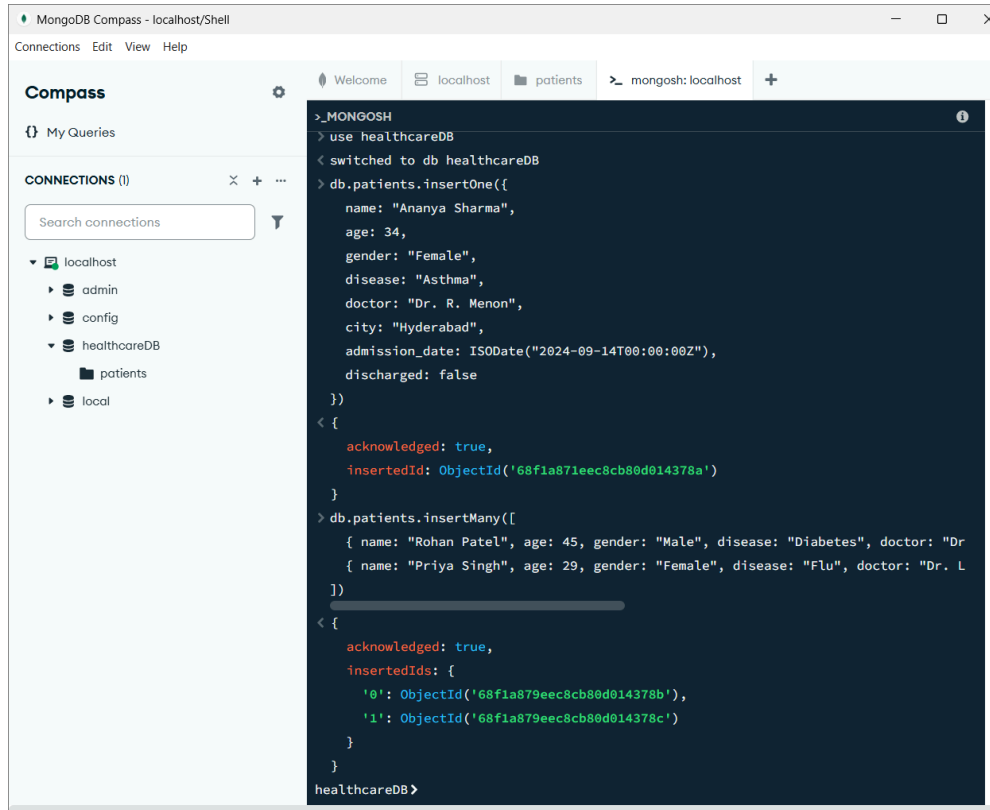
```
C: > Users > Bhaavya > Desktop > DBMS > insert_patients.py > ...
1 from faker import Faker
2 import csv
3 import random
4
5 fake = Faker()
6
7 with open("patients.csv", "w", newline="", encoding="utf-8") as csvfile:
8     fieldnames = ["name", "age", "gender", "disease", "doctor", "city", "admission_date", "discharged"]
9     writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
10    writer.writeheader()
11
12    for _ in range(100000):
13        writer.writerow({
14            "name": fake.name(),
15            "age": random.randint(1, 100),
16            "gender": random.choice(["Male", "Female", "Other"]),
17            "disease": random.choice(["Flu", "Diabetes", "Cancer", "COVID-19", "Asthma"]),
18            "doctor": fake.name(),
19            "city": fake.city(),
20            "admission_date": fake.date_this_decade(),
21            "discharged": random.choice([True, False])
22        })
23
24    print("Generated patients.csv with 100,000 records.")
25
```



CRUD Operations:

Performed Create, read, update, delete operations in MongoDB compass shell.

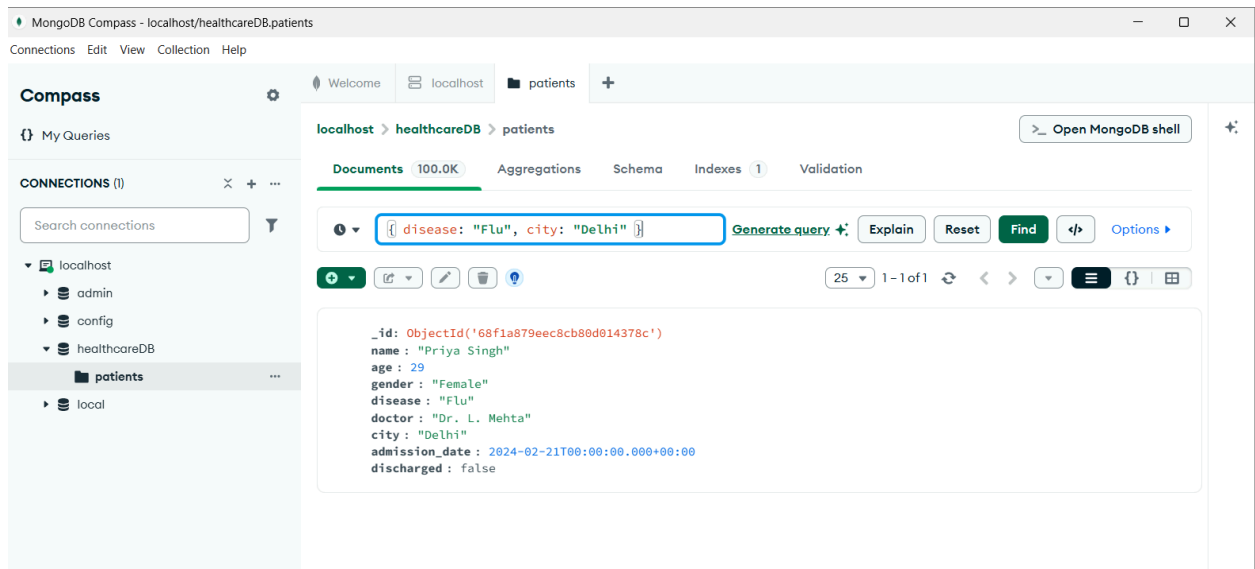
Create: Inserted a new patient record named Ananya Sharma into the database.



The screenshot shows the MongoDB Compass Shell interface. The left sidebar displays the 'connections' list with 'localhost' selected, showing the 'healthcareDB' database and the 'patients' collection. The main panel shows the MongoDB shell prompt with the following commands and output:

```
> use healthcareDB
switched to db healthcareDB
> db.patients.insertOne({
  name: "Ananya Sharma",
  age: 34,
  gender: "Female",
  disease: "Asthma",
  doctor: "Dr. R. Menon",
  city: "Hyderabad",
  admission_date: ISODate("2024-09-14T00:00:00Z"),
  discharged: false
})
{
  acknowledged: true,
  insertedId: ObjectId('68f1a871eec8cb80d014378a')
}
> db.patients.insertMany([
  { name: "Rohan Patel", age: 45, gender: "Male", disease: "Diabetes", doctor: "Dr. L. Mehta", city: "Delhi", admission_date: ISODate("2024-02-21T00:00:00Z"), discharged: false },
  { name: "Priya Singh", age: 29, gender: "Female", disease: "Flu", doctor: "Dr. L. Mehta", city: "Delhi", admission_date: ISODate("2024-02-21T00:00:00Z"), discharged: false }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68f1a879eec8cb80d014378b'),
    '1': ObjectId('68f1a879eec8cb80d014378c')
  }
}
healthcareDB>
```

Read: Retrieved patients with disease = flu and city = Delhi using filter queries.



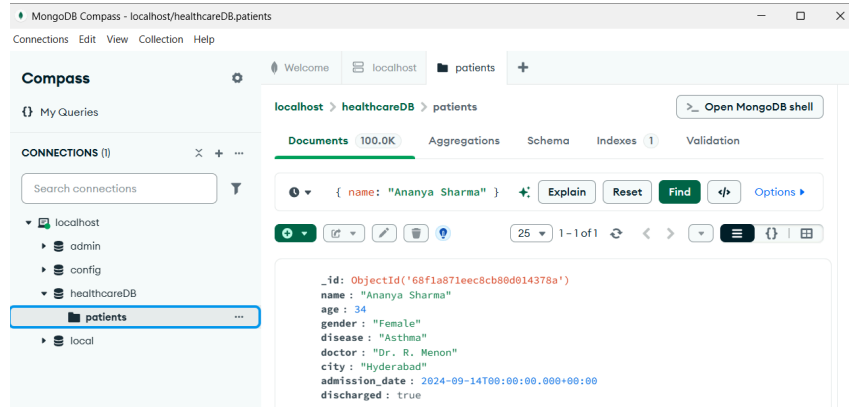
The screenshot shows the MongoDB Compass GUI. The left sidebar displays the 'connections' list with 'localhost' selected, showing the 'healthcareDB' database and the 'patients' collection. The main panel shows the 'patients' collection with a filter query applied: `{ disease: "Flu", city: "Delhi" }`. The query results are displayed in a table, showing the following document:

Document
<pre>{ "_id": ObjectId('68f1a879eec8cb80d014378c'), "name": "Priya Singh", "age": 29, "gender": "Female", "disease": "Flu", "doctor": "Dr. L. Mehta", "city": "Delhi", "admission_date": ISODate("2024-02-21T00:00:00Z"), "discharged": false }</pre>

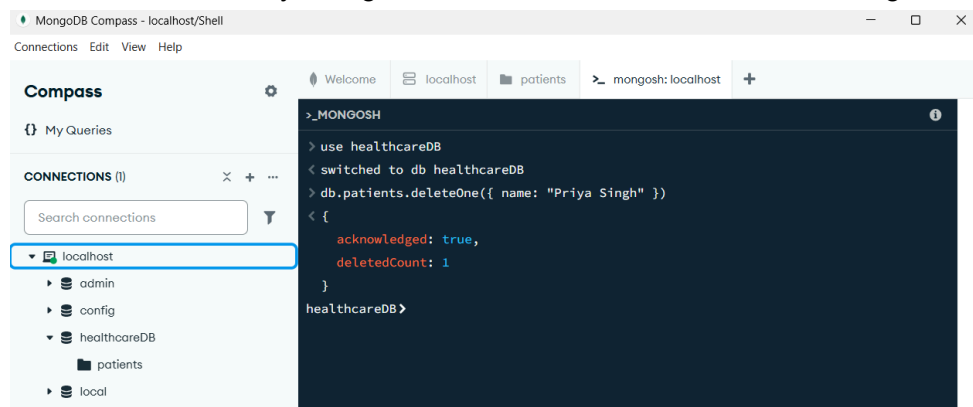
Update: Updated Ananya Sharma's record by setting discharged = true and verified through GUI filter.

```
>_MONGOSH

> use healthcareDB
< switched to db healthcareDB
> db.patients.updateOne(
  { name: "Ananya Sharma" },
  { $set: { discharged: true } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
healthcareDB>
```

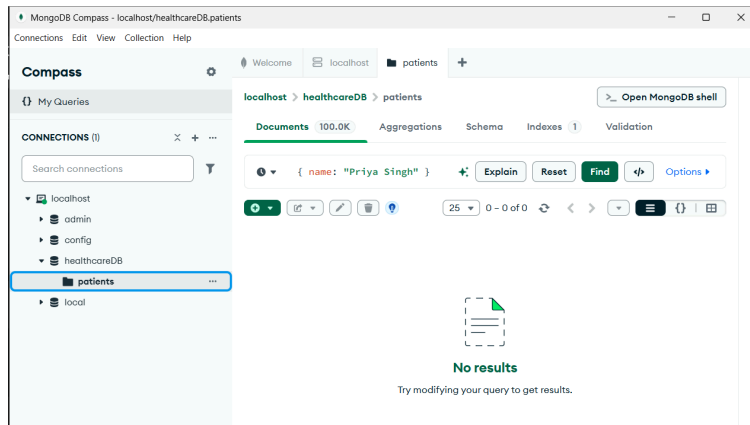


Delete: Removed Priya Singh's record and confirmed deletion through the Compass GUI.



```
>_MONGOSH

> use healthcareDB
< switched to db healthcareDB
> db.patients.deleteOne({ name: "Priya Singh" })
< {
  acknowledged: true,
  deletedCount: 1
}
healthcareDB>
```



No record found.

Indexing (MongoDB)

Created an index on the disease field to optimize query performance.

Execution time reduced from 84ms (no index) to 26ms (with index) since indexes speed up data lookups by reducing the number of scanned documents.

Without indexing

```
> use healthcareDB
switched to db healthcareDB
> db.patients.find({ disease: "Diabetes" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'healthcareDB.patients',
    parsedQuery: {
      disease: {
        '$eq': 'Diabetes'
      }
    },
    indexFilterSet: false,
    queryHash: '6f289774',
    planCacheShapeHash: '6f289774',
    planCacheKey: '933196f9',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExploreReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        disease: {
          '$eq': 'Diabetes'
        }
      },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 19833,
    executionTimeMillis: 84,
    totalKeysExamined: 0,
    totalDocsExamined: 100002,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',

```

executionTimeMillis = 84

Creation of index

localhost > healthcareDB > patients Open MongoDB shell

Documents 100.0K Aggregations Schema **Indexes 2** Validation

Create Index Refresh VIEWING INDEXES SEARCH INDEXES

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	1.0 MB	2 (since Fri Oct 17 2025)	UNIQUE	READY
> disease_1	REGULAR	544.8 KB	0 (since Fri Oct 17 2025)		READY

After index filter disease = 'diabetes'

Explain Plan

Explain provides key execution metrics that help diagnose slow queries and optimize index usage. [Learn more](#)

Visual Tree Raw Output Interpret for me

```
{
  "executionStats": {
    "executionSuccess": true,
    "nReturned": 19833,
    "executionTimeMillis": 26,
    "totalKeysExamined": 19833,
    "totalDocsExamined": 19833,
    "executionStages": {
      "isCached": false,
      "stage": "FETCH",
      "nReturned": 19833,
      "executionTimeMillisEstimate": 22,
      "works": 19834,
      "advanced": 19833,
      "needTime": 0,
      "needYield": 0,
      "saveState": 2,
      "restoreState": 2,
      "isEOF": 1,
      "docsExamined": 19833,
      "alreadyHasObj": 0,
      "inputStage": {
        "stage": "IXSCAN",
        "nReturned": 19833,
        "executionTimeMillisEstimate": 11,
        "works": 19834,
        "advanced": 19833,
        "needTime": 0,
        "needYield": 0,
        "saveState": 2,
        "restoreState": 2,
        "isEOF": 1,
        "keyPattern": { "disease": 1 },
        "indexName": "disease_1",
        "isMultiKey": false,
        "multikeyPaths": [ { "disease": 1 } ],
        "isUnique": false,

```

Query Performance Summary

- 19833 documents returned
- 19833 documents examined
- 26 ms execution time
- Is not sorted in memory
- 19833 index keys examined

Query used the following index:

disease_1

ExecutionTimeMillis = 26ms

Range Partitioning (MongoDB)

Partitioned data by age to distribute query load across smaller data segments.

Observed faster responses, baseline 76ms reduced to 31-70ms as queries accessed fewer records per partition.

Base line without age filter: 76ms

```
executionStats: {
  executionSuccess: true,
  nReturned: 19833,
  executionTimeMillis: 76,
  totalKeysExamined: 19833,
  totalDocsExamined: 19833,
```

With age < 20: 70ms

```
executionStats: {
  executionSuccess: true,
  nReturned: 3712,
  executionTimeMillis: 70,
  totalKeysExamined: 19833,
  totalDocsExamined: 19833,
```

With 20 <= age < 40: 70ms

```
executionStats: {
  executionSuccess: true,
  nReturned: 3712,
  executionTimeMillis: 70,
  totalKeysExamined: 19833,
  totalDocsExamined: 19833,
```

With 40 <= age < 60: 32ms

```
executionStats: {
  executionSuccess: true,
  nReturned: 3972,
  executionTimeMillis: 32,
  totalKeysExamined: 19833,
  totalDocsExamined: 19833,
```

With age >= 60: 31ms

```
executionStats: {
  executionSuccess: true,
  nReturned: 8220,
  executionTimeMillis: 31,
  totalKeysExamined: 19833,
  totalDocsExamined: 19833,
```

MySQL Partitioning

Retrieved all patient records from the table to verify successful data import before applying any partitioning or indexing.

33 • `SELECT * FROM patients;`

	id	name	age	gender	disease	doctor	city	admission_date	discharged
▶	1	Julie Good	49	Male	COVID-19	Katelyn Castro	South Elizabethburgh	2023-12-23	0
	2	Timothy Ryan	34	Male	COVID-19	Virginia Clark	East Steven	2022-04-09	0
	3	Melissa Henderson	40	Male	COVID-19	Linda Noble	Alexishaven	2020-05-14	0
	4	Brian Herrera	4	Female	COVID-19	Isaac Bray	Burtontown	2022-12-19	0
	5	Ryan Oliver	46	Female	Asthma	Thomas Rogers	Davisport	2024-01-17	0
	6	Danielle Mendez	95	Female	Flu	Marie Shaw	Hunterburgh	2023-09-27	0
	7	Margaret Johnson DDS	15	Female	Cancer	Leah Sloan	Troyview	2020-11-10	0
	8	Caroline McGuire	52	Male	Flu	Julie Carroll	East Tammypport	2024-03-11	0
	9	Mark Marsh Jr.	30	Female	Flu	Janet Norman	Whitemouth	2025-07-15	0
	10	Amanda Elliott	8	Other	Cancer	Bradley Wallace	Lynchbury	2021-08-10	0
	11	Ruth Garner	82	Other	Asthma	Anthony Rivera	Lake Andreaburgh	2022-05-19	0
	12	Sarah Page	15	Male	Flu	Timothy Bucha...	North Elizabeth	2022-04-25	0
	13	Elizabeth Hanson	38	Female	Diabetes	Courtney Hend...	Angelatown	2020-03-08	0
	14	Brooke Key	78	Female	Cancer	Kelly Snyder	Lake Matthew	2021-11-28	0
	15	Randy Knight	9	Female	Cancer	Christina Collins	North Jeffery	2022-02-04	0
	16	Sharon Savage	8	Male	Diabetes	Pamela Hayden	Justinburgh	2023-10-29	0

Created a new partitioned table using RANGE on the age column to group data into logical age ranges for efficient query access.

```

CREATE TABLE patients_partitioned (
  id INT AUTO_INCREMENT,
  name VARCHAR(100),
  age INT,
  gender VARCHAR(10),
  disease VARCHAR(50),
  doctor VARCHAR(100),
  city VARCHAR(50),
  admission_date DATE,
  discharged BOOLEAN,
  PRIMARY KEY (id, age)
)

PARTITION BY RANGE (age) (
  PARTITION p0 VALUES LESS THAN (20),
  PARTITION p1 VALUES LESS THAN (40),
  PARTITION p2 VALUES LESS THAN (60),
  PARTITION p3 VALUES LESS THAN MAXVALUE
);

TRUNCATE TABLE patients_partitioned;

INSERT INTO patients_partitioned(name, age, gender, disease, doctor, city, admission_date, discharged)
SELECT name, age, gender, disease, doctor, city, admission_date, discharged
FROM patients;

EXPLAIN SELECT * FROM patients_partitioned WHERE disease='Diabetes';

```

Displayed 100,000 total rows with filtered = 10%, showing index usage effectively narrows search space to matching disease records.

Result Grid Filter Rows: Export: Wrap Cell Content: FA											
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	patients_partitioned	p0,p1,p2,p3	ALL	NULL	NULL	NULL	NULL	100000	10.00	Using where

Range-Based Queries:

Created range partitions p0, p1, p2, p3 for age groups (<20, <40, <60, ≥60).

Each partition stores a subset of data, improving performance and manageability of large tables.

Age 20–39: 20,113 rows, filtered 11.11%

```

63 • EXPLAIN SELECT * FROM patients_partitioned WHERE age BETWEEN 20 AND 39;

```

Result Grid Filter Rows: Export: Wrap Cell Content: FA											
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	patients_partitioned	p1	ALL	NULL	NULL	NULL	NULL	20113	11.11	Using where

Age 40–59: 20,107 rows, filtered 11.11%

65 • `EXPLAIN SELECT * FROM patients_partitioned WHERE age BETWEEN 40 AND 59;`

Result Grid

Filter Rows:

Export: 

Wrap Cell Content: [⌂](#)

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	patients_partitioned	p2	ALL	NULL	NULL	NULL	NULL	20107	11.11	Using where

Age ≥ 60: 39,265 rows, filtered 33.33%

67 • `EXPLAIN SELECT * FROM patients_partitioned WHERE age >= 60;`

Result Grid

Filter Rows:

Export: 

Wrap Cell Content: 

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	patients_partitioned	p3	ALL	NULL	NULL	NULL	NULL	39265	33.33	Using where

Filtered percentages reflect the distribution of data across partitions, confirming that range partitioning reduces scan size and enhances performance.